

Relatório de Auditoria de Segurança

StudyAgent

Data: 02 de setembro de 2026

Escopo auditado: StudyAgent — backend FastAPI/Python (routers, agent/tools, tutor, security, core), frontend React/TypeScript/Vite, deploy (Dockerfile, docker-compose, systemd, CI GitHub Actions), config (permissions.json, .env/.env.example) e histórico git.

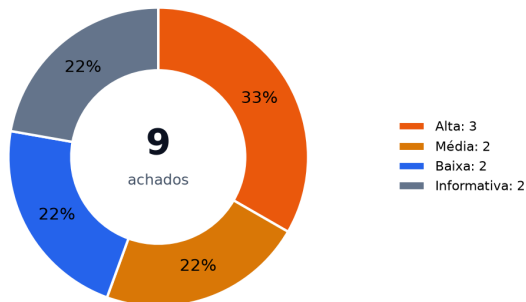
Nota metodológica (mapeamento das categorias para a stack)

O projeto é um app local single-user (SQLite, sem ORM; auth por PIN local via header X-StudyAgent-Pin; sem conceito de usuário/tenant). As categorias foram adaptadas: (1) 'Banco sem trava' → controle de quem acessa os dados da API, já que RLS/tenant não existe — o equivalente é a ausência de autenticação/autorização na leitura de documentos, conversas, perfil, faces e estatísticas. (2) 'Permissão definida no navegador' → cruzamento do painel de permissões/PIN no frontend com o que o backend realmente valida em cada rota sensível. (3) IDOR → não aplicável como multi-usuário; o equivalente (objeto por ID sem autorização) foi verificado em todos os handlers e é coberto pelo achado A1. (4) Chaves expostas → varredura do código, configs, deploy, histórico git e bundle dist do frontend. (5) Inputs sem tratamento → XSS no frontend/backend e SSRF via tools web.

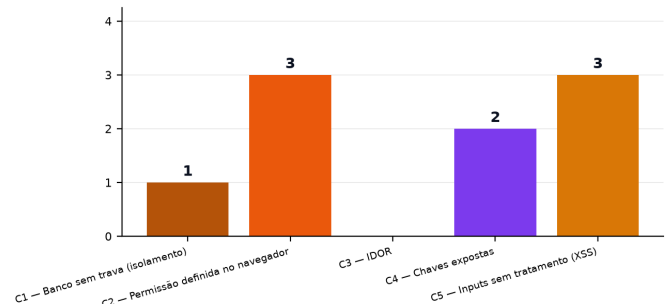
Resumo executivo

Foram verificados todos os routers do backend (chat, screen, exercises, documents, audio, audio_stream, facial, tutor, health), o pipeline do agente e tools, o frontend (todos os componentes), os arquivos de deploy e o histórico git. Resultado: **9 achados** (0 críticos, 3 altos, 2 médios, 2 baixos, 2 informativos) e **13 pontos fortes** confirmados, incluindo a ausência de SQL injection, de XSS clássico e de segredos no git.

Distribuição por severidade



Distribuição por categoria



Pontos fracos centrais (riscos)

1. Ausência total de autenticação na API → dados pessoais (documentos, conversas, biometria, perfil) legíveis sem credencial (A1).
2. Aprovação de automação sem PIN, contrariando a própria especificação do projeto; confirmação é só um botão no navegador (A2).
3. SSRF via web_search/open_url com permissão internet ativa por padrão e sem bloqueio de redes internas — explorável via prompt injection (A7).

Pontos fortes (protegido — com evidência)

- ✓ **SQL totalmente parametrizado.** `memory.py`, `tutor/*.py` e `export_import.py` usam apenas placeholders '?' — nenhuma concatenação de SQL no código (varredura por `execute(f.../format` retornou 0).
- ✓ **Calculadora AST-safe.** `calculator.py:17-33` avalia apenas árvore AST com allowlist de operações — sem eval de texto do usuário.
- ✓ **CSV export com mitigação de fórmula.** `export_import.py:20-28` prefixa ' em valores iniciados por =,+,-,@, , (CSV injection).
- ✓ **Sem XSS clássico no frontend.** nenhum `dangerouslySetInnerHTML/innerHTML/v-html/eval/new Function` em `frontend/src`; chat é renderizado como texto React (`ChatMessages.tsx:23`).
- ✓ **CORS restrito.** `main.py:61-66` permite apenas `localhost:5173` e `127.0.0.1:5173`.
- ✓ **Rate limiting.** `slowapi` em `/chat`, `/exercises/*`, `/flashcards/generate`, `/study-plans/generate`, `/audio/*`, `/actions`, `/face/*` (ex.: `chat.py:38`).
- ✓ **PIN com comparação constante e fail-closed.** `local_auth.py:36-46` usa `hmac.compare_digest`; sem config → 401 (mode deny).
- ✓ **Ferramentas de arquivo negadas por padrão.** `registered_tools.py` exige permissão 'filesystem' inexistente nas permissões (`permissions.py:23-32`) → sempre bloqueadas (fail-closed).
- ✓ **Upload restrito.** `documents.py:35-37` valida extensão (`.pdf/.txt/.md`) e salva com nome UUID; PDF é validado/extraído antes de persistir.
- ✓ **Content-Disposition sanitizada.** `documents.py:64` remove \ ; " CR LF do nome no header (sem header injection).
- ✓ **Bind local por padrão.** `systemd (scripts/*.service)` usa `--host 127.0.0.1`; `docker-compose` publica `127.0.0.1:8000/5173`.
- ✓ **Sem segredos no git/bundle.** `.env` gitignored; `git log -S` e `bundle dist` sem chaves/API keys (apenas hashes de integridade de `package-lock`).
- ✓ **Permissões de sensores no backend.** `microfone/câmera/tela` exigem `permissions.require` (`audio.py:30`, `facial.py:47/63/81/95/104`, `screen.py:44/86`).

Pontos fracos (riscos centrais)

- Sem barreira de autenticação entre consumidor local/remoto e a API (A1).
- Consentimento de automação validado apenas na UI (A2).
- Busca na web e abertura de URL sem restrição de destino (A7).
- Inconsistência de permissão no módulo de documentos (A3) e PIN em claro no disco (A5).
- Endurecimento ausente: sem validação de startup, sem security headers, rotas mortas (A6/A8/A9).

Achados detalhados por categoria

IDOR (C3): não aplicável como multi-usuário — não existe conceito de dono/tenant. O acesso a objetos por ID (*doc_id*, *session_id*, *deck_id*, *plan_id*, *card_id*, *proposals_id*, *error_id*, *name*) sem verificação de autorização é o equivalente e está coberto pelo achado A1, confirmado na revisão sistemática de todos os handlers.

A1	Alta	C1	backend/app/main.py:31,68-77 · routers	API sem autenticação expõe todos os dados pessoais e ações
A2	Alta	C2	backend/app/routers/tutor.py:288-303	Aprovação de automação dispensa PIN local (confirmação só no navegador)
A3	Média	C2	backend/app/routers/documents.py:53-72	Download de documento ignora a permissão <code>file_access</code>
A4	Informativa	C2	backend/app/routers/tutor.py:396-431	PIN é gate de ativação única, não revalidado no uso
A5	Média	C4	backend/.env:4	PIN de autenticação em claro no working tree
A6	Baixa	C4	backend/app/core/env_validation.py · docker-compose.yml:12-15	Sem validação de startup nem injeção de <code>STUDYGAGENT_PIN</code> no container
A7	Alta	C5	backend/app/tools/web_search.py:200-212	SSRF via <code>web_search/open_url</code> com permissão <code>internet</code> habilitada por padrão
A8	Baixa	C5	frontend/nginx.conf:1-15	Sem security headers (CSP, X-Content-Type-Options, X-Frame-Options) no nginx
A9	Informativa	C5	backend/app/routers/tutor.py:464-482	Decorator de rota aplicado a classe; funções não registradas (código morto)

Detalhamento dos achados

A1 · API sem autenticação expõe todos os dados pessoais e ações

Categoria: C1 — Banco sem trava (isolamento) **Severidade:** Alta **Referência:** backend/app/main.py:31,68-77 · routers

Descrição: Nenhuma dependência de autenticação é aplicada em `main.py` e a maioria dos handlers de rota não exige PIN. Qualquer requisição HTTP — de um processo local, página web maliciosa na mesma máquina, ou da rede caso a porta seja publicada — lê/grava documentos, conversas, perfil, faces e estatísticas. Equivalente adaptado de 'banco sem trava' num app single-user: não existe controle de quem acessa os dados.

Evidência: `main.py:31` ``app = FastAPI(...)``; routers incluídos sem dependency (linhas 70-77); ex.: `documents.py:53` ``@router.get('/documents/{doc_id}/file')`` serve o arquivo sem checar permissão nem credencial; `chat.py:68-75` ``/sessions`` e ``/sessions/{id}/messages`` devolvem histórico; `tutor.py:193` ``/profile``; `facial.py:92` ``/face/list``.

Por que é explorável: A única proteção do projeto é o PIN em 3 rotas de ativação de permissão. Dados pessoais (documentos anexados, conversas, biometria facial, perfil) ficam acessíveis a qualquer chamada sem credencial. O Dockerfile:30 sobe `uvicorn` com ``--host 0.0.0.0``; o compose faz bind em 127.0.0.1 (por padrão), mas um deploy com porta publicada expõe tudo à LAN.

Impacto: Exfiltração de dados pessoais; manipulação do estado da aplicação sem autorização.

Sugestão de correção: Adicionar camada de auth local (ex.: token bearer/PIN convertido em sessão `httpOnly`, ou header fixo trocado por login) aplicada globalmente via dependency no app; manter bind 127.0.0.1; nunca publicar a porta sem auth.

Critérios de aceite: (1) Todos os endpoints `/api/*` exigem um token/sessão válidos (teste automatizado). (2) Falha de auth → 401 e log. (3) Dockerfile/compose não publicam a API na rede sem auth.

A2 · Aprovação de automação dispensa PIN local (confirmação só no navegador)

Categoria: C2 — Permissão definida no navegador **Severidade:** Alta **Referência:** backend/app/routers/tutor.py:288-303

Descrição: `POST /api/actions/{proposal\_id}/approve` e `/reject` executam sem `require\_pin`. O docstring de local_auth.py:6-7 afirma que aprovar/rejeitar propostas de automação DEVE exigir PIN. O frontend ActionConfirm.tsx:49-54 exibe o botão '✓ executar' sem pedir PIN — o 'gate' é apenas a UI.

Evidência: tutor.py:288-294 `@router.post('/actions/{proposal\_id}/approve')` ... return automation.approve(proposal_id) — sem require_pin; tutor.py:297-303 idem para reject. automation.py:55-73 aprova e executa a proposta. ActionConfirm.tsx:49-54 mostra os botões de confirmação sem exigir PIN.

Por que é explorável: O modelo documentado ('PIN garante consentimento') é viável apenas se o servidor validar o privilégio em toda via de concessão. Aqui a ação é grampeável por um request simples (POST sem body → sem preflight CORS), permitindo a uma página local maliciosa aprovar propostas (open_url/web_search) sem interação.

Impacto: Execução de ações propostas pelo agente sem consentimento do usuário (CSRF local + falha de autorização).

Sugestão de correção: Exigir `require\_pin` em approve/reject (e propor CSRF token/check de header customizado para requests simples). Manter alinhado ao docstring de local_auth.

Critérios de aceite: (1) require_pin presente em approve e reject. (2) Teste de API ativa sem PIN → 401. (3) Frontend solicita o PIN antes de aprovar/rejeitar.

A3 · Download de documento ignora a permissão file_access

Categoria: C2 — Permissão definida no navegador **Severidade:** Média **Referência:** backend/app/routers/documents.py:53-72

Descrição: `GET /api/documents/{doc\_id}/file` não chama `permissions.require('file\_access')`, enquanto os endpoints irmãos audio/plan (documents.py:92-94) e audio (documents.py:105-107) exigem. Desativar 'file_access' na UI não impede o download do documento.

Evidência: documents.py:57-72 (handler sem checagem) vs documents.py:92,105 (permissions.require('file_access')).

Por que é explorável: Inconsistência de autorização: o gate exibido no navegador (painel de permissões) não corresponde à proteção do servidor nesta rota de leitura de conteúdo.

Impacto: Conteúdo do documento legível mesmo com a permissão de acesso a arquivos revogada.

Sugestão de correção: Adicionar `permissions.require('file\_access')` no início do handler document_file (ou mover para um helper comum com os demais).

Critérios de aceite: (1) Com file_access=false, GET /documents/{id}/file → 403. (2) Teste cobre as 3 rotas de documento de forma consistente.

A4 · PIN é gate de ativação única, não revalidado no uso

Categoria: C2 — Permissão definida no navegador **Severidade:** Informativa **Referência:** backend/app/routers/tutor.py:396-431

Descrição: O PIN (require_pin) é exigido apenas no momento de ATIVAR uma permissão perigosa. Uma vez ativa, nenhum uso da capability revalida o PIN. Hoje nenhuma ferramenta usa command_execution/mouse_control/keyboard_control (nenhuma registrada em registered_tools.py), então o impacto atual é baixo, mas a arquitetura herdaria o risco de qualquer ferramenta futura.

Evidência: tutor.py:399-400 `if body.value and name in DANGEROUS\_PERMISSIONS: require\_pin(request)`; registros em registered_tools.py usam apenas 'internet' e 'filesystem'.

Por que é explorável: Modelo presume que a ativação é o momento de consentimento; um único request de ativação (com PIN vazado/observado) deixa o recurso disponível sem novo desafio.

Impacto: Baixo hoje; médio se for adicionada ferramenta de execução regida só pela permissão ativada.

Sugestão de correção: Para capabilities críticas (command_execution), revalidar o PIN no uso (ou exigir timeout de reconsentimento). Documentar o modelo.

Critérios de aceite: (1) Ferramenta crítica nova revalida PIN ou possui timeout de consentimento. (2) README descreve o modelo de reconsentimento.

A5 · PIN de autenticação em claro no working tree

Categoria: C4 — Chaves expostas **Severidade:** Média **Referência:** backend/.env:4

Descrição: `STUDYAGENT\_PIN=c59dc128b5626deba268410a` existe no arquivo backend/.env do diretório de trabalho. O arquivo é gitignored (não rastreado) e o valor não está no histórico do git — verificado —, mas o segredo trafega em claro no disco e pode vaziar via `git add -f`, backup/manual ou tar/zip recursivo da pasta.

Evidência: backend/.env:4 `STUDYAGENT\_PIN=c59dc128b5626deba268410a`; .gitignore:17 `backend/.env`; risk-register.md:18 (R4) já documenta o risco.

Por que é explorável: Qualquer segredo em claro no disco aumenta a superfície (malware local lê o arquivo). O próprio risk-register reconhece R4 como risco residual.

Impacto: Comprometimento do PIN de consentimento se o arquivo vazar (backup/commit incladvertido).

Sugestão de correção: Rotacionar o PIN; injetar via environment variables do deploy/service; garantir que backup.sh nunca inclua .env; adicionar guard em script de backup/empacotamento.

Critérios de aceite: (1) PIN rotacionado e nova variante fora do working tree rastreável. (2) backup.sh e scripts de empacotamento excluem .env (teste). (3) Grep de segredo conhecido retorna 0 em artefatos de entrega.

A6 · Sem validação de startup nem injeção de STUDYAGENT_PIN no container

Categoria: C4 — Chaves expostas **Severidade:** Baixa **Referência:** backend/app/core/env_validation.py · docker-compose.yml:12-15

Descrição: `validate\_environment()` não valida presença/fortaleza de STUDYAGENT_PIN. O .env.example deixa o valor vazio por padrão (fail-closed correto para as operações perigosas, que retornam 401), mas o docker-compose não injeta a variável — no container todas as operações privilegiadas ficam permanentemente 401 (recurso inerte) sem nenhum aviso claro.

Evidência: env_validation.py:79-92 (checks de python/tesseract/ollama/disco/dirs, sem PIN); docker-compose.yml:12-15 (environment sem STUDYAGENT_PIN).

Por que é explorável: Padrão vazio é seguro por omissão, mas a falta de verificação de startup permite deploy silenciosamente sem a capacidade esperada e sem alerta.

Impacto: Baixo (disponibilidade/observabilidade); sem vazamento, mas configuração ambígua.

Sugestão de correção: Avisar em validate_environment() quando STUDYAGENT_PIN estiver ausente em modo não-interativo; injetar a variável no compose a partir do ambiente do host.

Critérios de aceite: (1) Log de aviso/warning no startup quando PIN ausente. (2) Compose injeta STUDYAGENT_PIN (ou bloquear modo sem PIN).

A7 · SSRF via web_search/open_url com permissão internet habilitada por padrão

Categoria: C5 — Inputs sem tratamento (XSS) **Severidade:** Alta **Referência:** backend/app/tools/web_search.py:200-212

Descrição: `fetch\_page` valida apenas o prefixo `^https?://` (web_search.py:202) e executa `requests.get(url)` seguindo redirecionamentos (linha 204), sem vetar endereços privados/link-local/metadata (127.0.0.0/8, 169.254.169.254, 10/8, 172.16/12, 192.168/16, ::1). A ferramenta open_url (registered_tools.py:79-80) e o próprio web_search_tool, que baixa cada resultado (registered_tools.py:43), usam esse código. A permissão que as gateia (internet) está ATIVA por padrão (permissions.py:28, config/permissions.json:6) sem exigir PIN.

Evidência: web_search.py:200-212; registered_tools.py:37-58 (web_search_tool) e 79-80 (open_url_tool); permissions.py:28 `internet: True`.

Por que é explorável: ('Gatilhos: (1) prompt injection — conteúdo de página web ou do próprio chat instrui o LLM a chamar open_url com URL interna; (2) o modelo escolhe URLs de resultados; (3) o usuário pede abertura de URL. O texto baixado volta para a resposta do chat (exfiltração de serviços internos/metadados).')

Impacto: Acesso e exfiltração de endpoints internos (Ollama, API local, metadados de cloud para o Llama/host se hospedado).

Sugestão de correção: Permitir apenas destinos públicos: resolver DNS e bloquear IPs privados/link-local/metadata; não seguir redirects (allow_redirects=False) reavaliando cada destino; por default `internet` = False (opt-in com PIN).

Critérios de aceite: (1) Teste: open_url para 127.0.0.1/169.254.169.254/10.x → erro. (2) Redirect para IP privado → bloqueado. (3) internet default = false (e 401/403 sem PIN).

A8 · Sem security headers (CSP, X-Content-Type-Options, X-Frame-Options) no nginx

Categoria: C5 — Inputs sem tratamento (XSS) **Severidade:** Baixa **Referência:** frontend/nginx.conf:1-15

Descrição: A configuração de serviço do frontend não envia Content-Security-Policy nem demais headers de endurecimento. Defesa em profundidade ausente.

Evidência: frontend/nginx.conf:1-15 (apenas listen/root/try_files/proxy_pass).

Por que é explorável: Sem CSP, qualquer futuro vetor XSS (ou script vindo do futuro renderizador de markdown) teria caminho livre para exfiltração do PIN em sessionStorage.

Impacto: Baixo hoje (sem XSS encontrado), mitigação faltante.

Sugestão de correção: Adicionar headers no bloco server (CSP default-src 'self'; X-Content-Type-Options: nosniff; X-Frame-Options: DENY; Referrer-Policy).

Critérios de aceite: (1) curl retorna os headers. (2) CSP não quebra assets do bundle existente.

A9 · Decorator de rota aplicado a classe; funções não registradas (código morto)

Categoria: C5 — Inputs sem tratamento (XSS) **Severidade:** Informativa **Referência:** backend/app/routers/tutor.py:464-482

Descrição: `@router.post('/errors/review-topic')` decora uma classe (`ReviewTopicRequest`) e `review_topic_errors.generate_flashcards_from_errors` não possuem decorator — não expõem rotas como previsto. Sem impacto de segurança direto, mas rotas que parecem ativas não existem e qualquer handler futuro copiado deste padrão pode escapar do controle.

Evidência: tutor.py:464-466 (`@router.post('/errors/review-topic')` sobre class ...) e 475-482 (decorator sobre class; funções sem decorator).

Por que é explorável: Padrão frágil que mascara a superfície real de exposição da API; indicador de revisão.

Impacto: Informativo (funcional); superfície esperada não condiz com a real.

Sugestão de correção: Mover os decorators para as funções; remover/reexpor conforme intenção; adicionar teste de rota.

Critérios de aceite: (1) Routes `/errors/review-topic` e `/flashcards/generate-from-errors` respondem conforme documentado (ou são removidas). (2) Teste de rota cobre o endpoint.

Recomendações priorizadas

P1 — Autenticação na API

Aplicar dependency global de auth local (token/sessão httpOnly derivada do PIN) em todos os `/api/*`; nunca publicar a porta sem auth (A1).

P1 — PIN em aprovação de automação

require_pin em `/actions/{id}/approve` e `/reject` + CSRF/check de header customizado; alinhar ao docstring de local_auth (A2).

P1 — Endurecer fetch de URLs

Allowlist pública com bloqueio de IPs privados/link-local/metadata, sem seguir redirects, e internet default = False (opt-in com PIN) (A7).

P2 — Consistência de file_access

Exigir file_access em GET `/documents/{id}/file` (A3).

P2 — Segredo do PIN

Rotacionar o PIN; injetar via env do deploy/service; blindar backup/empacotamento contra inclusão de .env (A5).

P2 — Validação de startup

Avisar quando STUDYAGENT_PIN ausente; injetar a variável no compose (A6).

P3 — Security headers

CSP + X-Content-Type-Options + X-Frame-Options + Referrer-Policy no nginx (A8).

P3 — Sanar rotas mortas

Corrigir decorators de `/errors/review-topic` e `/flashcards/generate-from-errors` (A9).

P3 — Modelo de reconsentimento

Para capability crítica futura: revalidar PIN no uso/timeout; documentar (A4).

Issues para o GitHub

Blocos prontos para copiar e colar na aba Issues. Cada bloco contém título, labels sugeridas, problema/explorabilidade, evidência (arquivo:linha), impacto, correção e critérios de aceite verificáveis. Os achados triviais foram agrupados numa única issue de hardening para evitar spam.

ISSUE 1

```
--- ISSUE 1 ---
### [Segurança] API do StudyAgent sem autenticação expõe dados pessoais e ações privilegiadas
Labels: `security, alta`

## Problema
A API FastAPI (backend/app/main.py) não aplica nenhuma dependência de autenticação e, exceto por 3 rotas de ativação de permissão, nenhum handler valida credencial ou PIN. Endpoints como `GET /api/documents/{doc_id}/file`, `GET /api/sessions`, `GET /api/sessions/{session_id}/messages`, `GET /api/profile` e `GET /api/face/list` são legíveis por qualquer requisição.

## Por que é explorável
Um processo ou página local maliciosa (ou a rede, se a porta for publicada) lê todos os dados pessoais sem credencial. `Dockerfile:30` sobe uvicorn com `--host 0.0.0.0`.

## Evidência
`backend/app/main.py:31,70-77` (routers sem dependency) · `backend/app/routers/documents.py:53` · `backend/app/routers/chat.py:68-75` · `backend/app/routers/facial.py:92`.

## Impacto
Exfiltração de documentos/conversas/perfil/biometria; manipulação de estado sem autorização.

## Sugestão de correção
Adicionar token/sessão local (ex.: PIN convertido em cookie httpOnly ou header bearer) aplicado globalmente via dependency; manter bind 127.0.0.1; não publicar a porta sem auth.

## Critérios de aceite
- [ ] Todos os endpoints /api/* exigem autenticação (teste automatizado por rota).
- [ ] Sem credencial → 401 + log.
- [ ] Dockerfile/compose não expõem a API em 0.0.0.0 sem auth.
--- FIM ISSUE 1 ---
```

ISSUE 2

```
--- ISSUE 2 ---
### [Segurança] Aprovação de automação dispensa PIN local – 'confirmação' só no navegador
Labels: `security, alta`

## Problema
`POST /api/actions/{proposal_id}/approve` e `/reject` (backend/app/routers/tutor.py:288-303) não chamam `require_pin`, contrariando a especificação em `backend/app/security/local_auth.py:6-7`, que determina PIN para aprovar/rejeitar automação. O frontend exibe '✓ executar' sem exigir PIN (frontend/src/components/ActionConfirm.tsx:49-54).

## Por que é explorável
A 'confirmação' do usuário é apenas um clique na UI. O endpoint aceita POST simples sem body (sem preflight CORS), permitindo a uma página local maliciosa aprovar propostas (ex.: open_url/web_search) sem interação.

## Evidência
`backend/app/routers/tutor.py:288-294 e 297-303` · `backend/app/tutor/automation.py:55-73` · `frontend/src/components/ActionConfirm.tsx:49-54`.

## Impacto
Execução de ações do agente sem consentimento (CSRF local + falha de autorização).

## Sugestão de correção
Exigir `require_pin` em approve/reject; adicionar check de header customizado/CSRF token para requests simples; pedir PIN na UI antes de confirmar.

## Critérios de aceite
- [ ] require_pin presente em approve e reject.
- [ ] Chamada sem PIN → 401 (teste).
```

```
- [ ] Frontend solicita PIN antes de aprovar/rejeitar.  
--- FIM ISSUE 2 ---
```

ISSUE 3

```
--- ISSUE 3 ---  
### [Segurança] SSRF via web_search/open_url com permissão internet habilitada por padrão  
Labels: `security, alta`  
  
## Problema  
`fetch_page` (backend/app/tools/web_search.py:200-212) valida apenas o prefixo `https?://` e chama  
`requests.get(url)` seguindo redirects, permitindo destinos internos (127.0.0.1, 10.x, 172.16/12,  
192.168.x, 169.254.169.254, ::1). As ferramentas `open_url` (registered_tools.py:79-80) e `web_search`  
(registered_tools.py:43, que baixa cada resultado) usam esse fluxo; `internet` está ativa por padrão  
(permissions.py:28) e não exige PIN para desativar/ativar.  
  
## Por que é explorável  
Gatilhos: prompt injection (conteúdo web/chat instrui o LLM a abrir URL interna), o agente escolhendo URLs  
de resultados, ou o usuário pedindo abertura de URL. O conteúdo baixado volta na resposta do chat  
(exfiltração).  
  
## Evidência  
`backend/app/tools/web_search.py:200-212` · `backend/app/core/registered_tools.py:37-58,79-80` ·  
`backend/app/security/permissions.py:28` · `config/permissions.json:6`.  
  
## Impacto  
Leitura/exfiltração de endpoints internos (Ollama, API local, metadados de cloud).  
  
## Sugestão de correção  
Bloquear IPs privados/link-local/metadata após resolução DNS; `allow_redirects=False` reavaliando cada  
destino; `internet` por padrão = false com opt-in via PIN.  
  
## Critérios de aceite  
- [ ] open_url/web_search para 127.0.0.1, 169.254.169.254, 10.x → erro.  
- [ ] Redirect para IP privado → bloqueado.  
- [ ] Default de 'internet' = false; ativação exige PIN.  
--- FIM ISSUE 3 ---
```

ISSUE 4

```
--- ISSUE 4 ---  
### [Segurança] Download de documento ignora a permissão file_access  
Labels: `security, media`  
  
## Problema  
`GET /api/documents/{doc_id}/file` (backend/app/routers/documents.py:53-72) não chama  
`permissions.require('file_access')`, ao contrário dos irmãos `/audio/plan` (linha 92) e `/audio` (linha  
105).  
  
## Por que é explorável  
O painel de permissões do navegador comunica 'file_access' como controle, mas a rota de leitura de  
conteúdo não aplica o mesmo controle no servidor.  
  
## Evidência  
`backend/app/routers/documents.py:53-72` vs `documents.py:92,105`.  
  
## Impacto  
Documento legível mesmo com file_access revogada.  
  
## Sugestão de correção  
Adicionar `permissions.require('file_access')` ao handler document_file (helper comum).  
  
## Critérios de aceite  
- [ ] file_access=false → GET /documents/{id}/file retorna 403.  
- [ ] Teste cobre as 3 rotas de documento.  
--- FIM ISSUE 4 ---
```

ISSUE 5

```
--- ISSUE 5 ---  
### [Segurança] PIN de autenticação em claro no working tree (backend/.env)
```

Labels: `security, media`

Problema

`backend/.env:4` contém `STUDYAGENT\_PIN=c59dc128b5626deba268410a`. O arquivo é gitignored e não consta do histórico (verificado), mas o segredo fica em claro no disco.

Por que é explorável

Vazamento por `git add -f`, backup/manual ou tar/zip recursivo da pasta expõe o consentimento. O risk-register já marca R4 como risco residual.

Evidência

`backend/.env:4` · `.gitignore:17` · `risk-register.md:18`.

Impacto

Comprometimento do PIN de consentimento se o arquivo vazar.

Sugestão de correção

Rotacionar o PIN; injetar via variáveis de ambiente do deploy; blindar backup/empacotamento contra inclusão de .env.

Critérios de aceite

- [] PIN rotacionado e variante antiga não rastreável.
- [] backup.sh e empacotamento excluem .env (teste).
- [] Grep do segredo conhecido retorna 0 nos artefatos.

--- FIM ISSUE 5 ---

ISSUE 6

--- ISSUE 6 ---

[Segurança] Hardening complementar: validação de PIN no startup, reconsentimento e security headers

Labels: `security, baixa`

Problema

Três itens menores reunidos:

1. `validate\_environment()` (backend/app/core/env_validation.py:79-92) não avisa quando STUDYAGENT_PIN está ausente; o compose não injeta a variável (docker-compose.yml:12-15), deixando o container com operações privilegiadas permanentemente 401 sem alerta (A6).
2. O PIN é gate de ATIVAÇÃO única; o uso da capability não revalida (tutor.py:399-400). Impacto atual baixo (nenhuma ferramenta usa command_execution hoje), mas arriscado para ferramentas futuras (A4).
3. frontend/nginx.conf não envia CSP nem X-Content-Type-Options/X-Frame-Options (A8).

Sugestão de correção

- Avistar/alertar quando PIN ausente; injetar a variável no compose.
- Revalidar PIN (ou timeout de reaconsentimento) em capability crítica; documentar o modelo.
- Adicionar headers de segurança (CSP default-src 'self', nosniff, DENY, Referrer-Policy).

Critérios de aceite

- [] Aviso de PIN ausente no startup.
- [] Compose injeta STUDYAGENT_PIN.
- [] Documentação do modelo de reaconsentimento.
- [] Headers CSP/nosniff/X-Frame presentes (curl).

--- FIM ISSUE 6 ---